

1.Brève introduction au protocole (histoire, fonctionnement, ...)

HTTP est le protocole sur lequel repose le World Wide Web. C'est un protocole d'échange d'information, justifiant le mode d'échange de messages entre le client et le serveur. Créé à la fin des années 1980, début des années 1990 par Tim Berners-Lee et son équipe au CERN, il a donné naissance au Web, à internet dans le format que l'on connaît aujourd'hui.

La première version, "HTTP/0.9", était très simple : elle ne permettait que de récupérer des documents HTML composés de texte brut, sans en-tête, sans métadonnées ni gestion d'erreur. La seule commande envoyée par le client, à l'initiative du protocole, était "GET" suivie du nom du fichier demandé pour permettre au serveur de renvoyer le contenu du fichier. Une façon de faire très basique rapidement saturée avec le développement du Web. C'est ainsi qu'en 1996, "HTTP/1.0" introduit une version beaucoup plus élaborée du protocole avec la possibilité de transférer des contenus différents comme des images, sons, vidéos,... Le recours aux header permet aussi le transport d'informations contextuelles — type de contenu, taille des données, langue,... Il y a aussi des codes d'état (voir point 8) énonçant le résultat de la requête.

Une année plus tard, le protocole fait désormais l'objet d'un standard officiel, qui est encore aujourd'hui le plus largement employé, "HTTP/1.1". Les performances et la fiabilité ont été ainsi considérablement améliorées. La version 1.1 introduit la notion de connexion persistante, permettant d'envoyer plusieurs requêtes successives sur la même connexion TCP sans qu'il ne soit nécessaire de la rouvrir à chaque fois et ainsi de réduire les temps de latence. Elle propose également les mécanismes de gestion de cache, de requête partielle, de compression des données, ainsi qu'un meilleur contrôle du trafic via l'en-tête.

Avec la montée en puissance et des demandes des applications Web complexes, on arrive cependant à bout de souffle avec "HTTP/1.1". En 2015, "HTTP/2", tirant parti du protocole expérimental SPDY de Google, voit le jour en prenant la même architecture, mais révolutionne le mode d'échanges des données. "HTTP/2" prend

surtout appui sur un mode de communication binaire à la place du texte brut rendant les échanges plus rapidement analysables, moins sujet à erreurs, et surtout il supporte le multiplexage, permettant d'envoyer plusieurs requêtes et plusieurs réponses simultanées sur une même connexion TCP pour éviter le blocage pour cause de file d'attente que qu'il y avait sous "HTTP/1.1".

Concernant le fonctionnement d'HTTP, on peut dire qu'il est basé sur un modèle client-serveur. Le client est l'initiateur de la communication qui envoie une requête HTTP qu'il rédige en trois parties : une ligne de requête, des en-têtes et éventuellement un corps avec le message. Le serveur répond à cette requête avec une réponse HTTP qui comporte une ligne de statut, des en-têtes et le contenu demandé. Ces échanges se font sur le port 80 et le transport des données est réalisé par le protocole TCP/IP.

Une des caractéristiques d'HTTP est son caractère stateless. Autrement dit, le protocole ne garde rien en mémoire. Mais, lors d'une connexion, des requêtes HTTP sont reliées et requièrent des sessions, des tokens, et des cookies pour maintenir des passages d'un état à l'autre.

2.Interactions entre client et serveur

Si on se penche plus précisément sur les interactions client-serveur, voici ce qu'il se passe.

Lorsqu'un utilisateur saisit une adresse web dans son navigateur, celui-ci devient un client HTTP. Le navigateur traduit l'adresse, l'url en une requête adressée au serveur qui héberge le site. Pour cela, il utilise le protocole TCP/IP pour établir une connexion au port 80. Une fois la connexion ouverte, le client envoie une requête HTTP structurée, composée de plusieurs parties. La première ligne, appelée ligne de requête, contient la méthode HTTP, comme GET, POST, PUT, DELETE ou HEAD (on les verra plus tard point 9) et l'URI qui identifie la ressource demandée ainsi que la version du protocole. Après il y a les en-têtes de requête, qui fournissent des informations contextuelles. Le type de données accepté, la langue de l'utilisateur, le navigateur utilisé ou encore des identifiants de session. Certaines requêtes,

notamment celles utilisant POST ou PUT, comportent également un body contenant des données à envoyer au serveur comme un fichier.

Une fois la requête reçue, le serveur HTTP la traite. Il identifie la ressource demandée, exécute éventuellement des scripts ou des programmes, puis, prépare une réponse HTTP. Cette réponse est structurée elle aussi en plusieurs parties. La première ligne, appelée ligne de statut, indique la version du protocole et un code d'état décrivant le résultat du traitement. Par exemple, 200 OK signifie que tout s'est bien passé, 404 Not Found indique que la ressource n'existe pas, et 500 Internal Server Error signale une erreur du côté du serveur. Les en-têtes de réponse précisent des informations sur le contenu retourné. Notamment sa taille ou encore des cookies à enregistrer. Enfin, le corps de la réponse contient le contenu proprement dit. Une page HTML, une image, du texte, ou encore un document JSON selon la demande initiale.

Le client reçoit cette réponse, interprète le code d'état et affiche ou traite le contenu reçu. Par exemple, dans le cas d'une page web, le navigateur analyse le code HTML, télécharge les ressources supplémentaires mentionnées. Ça peut être des images, feuilles CSS, scripts JavaScript, et rend la page visible à l'écran. Chaque ressource liée à la page déclenche à son tour une nouvelle requête HTTP, sur la même connexion si le protocole utilisé est HTTP/1.1 ou une version ultérieure comme expliqué dans le point 1.

En somme :

1. Le client établit une connexion TCP avec le serveur.
2. Il envoie une requête HTTP structurée selon le protocole.
3. Le serveur traite la requête et renvoie une réponse contenant le code d'état, les en-têtes et le corps de message.
4. Le client interprète la réponse, l'affiche ou agit en conséquence.
5. La connexion peut ensuite être fermée ou maintenue ouverte.

3. Structure d'une requête HTTP (headers, méthodes, version, body, query parameters, ...)

```
POST /utilisateurs/creer HTTP/1.1
Host: api.exemple.com
Content-Type: application/json
Accept: application/json
User-Agent: MonApp/1.0
Authorization: Bearer <votre_token_secret>
Content-Length: 39

{"nom": "Alex", "role": "admin"}
```

Nous avons ici un exemple de requête http. On voit qu'il est composé de 3 parties distinctes. La Request-line, les headers et le body.

La Request line.

C'est la première ligne et elle est elle-même composée de 3 éléments.

La première partie s'appelle de verbe, ici c'est post. C'est basiquement l'action que veut faire le client. On reviendra plus tard sur les plus courants.

La deuxième partie est l'URI et les query parameters. C'est la cible de la requête. Il peut y avoir un path mais aussi des enchainement de & et de ?. Ce sont des paires de clés valeurs qui servent notamment à trier une demande get.

La troisième partie indique la version du protocole utilisée.

Les headers.

Ce sont des paires de clé,valeurs qui donnent des instructions et des données au serveur. On va les voir plus en profondeur sur le point 7, je ne vais pas aller en profondeur ici.

Le body.

Il n'est pas nécessaire dans tous les cas. Sa présence dépend du verbe utilisé et sa forme dépend du content-type. Il peut s'agir d'un json, d'un texte brute ou encore d'une url encodé.

4. Structure d'une URL

```
https://jean:secret@www.exemple.com:8080/chemin/vers/doc.html?  
q=motcle&filtre=on#section-2
```

Nous avons ici une URL complexe reprenant quasi toutes les composantes possibles. Nous allons la décortiquer.

Scheme.

C'est le protocole que doit utiliser le client pour accéder à la ressource. Ici, https. Ça peut aussi être http, ftp,...

Authority.

Cette partie est décomposée en 3 parties.

La première partie est le userinfo. Ici c'est jean:secret@. Ici le mot de passe est visible en clair, c'est une faille de sécurité majeur. Très peu commun.

La deuxième partie est le host. Ici www.exemple.com. C'est tout simplement le nom de domaine que l'on essaye de contacter. Il sera interprété par le dns pour avoir l'ip.

La troisième partie nous donne tout simplement le port. Ici 8080.

Path.

C'est la ressource que l'on veut atteindre dans le serveur. Ici /chemin/vers/doc.html.

Query.

Cette partie fourni des paramètres dynamiques. Comme précédemment expliqué, elle commence par ?. Puis nous avons des clés/valeurs séparées par des &.

Fragment

On l'appelle aussi l'anchor. Il commence toujours par un #. Il n'est pas envoyé au serveur, il est uniquement utile au navigateur. Ici, il demande au navigateur de faire défiler la page jusqu'à un certain id demandé.

5. Fonctionnement d'un formulaire HTML

Un formulaire HTML est une partie d'une page web qui permet à un utilisateur d'entrer et de soumettre des données à un site ou une application. Il construit automatiquement une requête http en quelque sorte. Voici un exemple :

```
<html>
  <head>
    <title>Cours HTML et CSS</title>
    <meta charset= "utf-8">
    <link rel="stylesheet" href="cours.css">
  </head>

  <body>
    <form method="post" action="form.php">
      <div class="champ">
        <label for="pseudo">Entrez un pseudonyme : </label>
        <input type="text" id="pseudo" name="pseudo">
      </div>
      <div class="champ">
        <label for="pass">Entrez un mot de passe :</label>
        <input type="password" id="pass" name="pass">
      </div>
      <div class="champ">
        <label for="mail">Entrez un mail :</label>
        <input type="email" id="mail" name="mail">
      </div>
      <div class="champ">
        <input type="submit" value="Envoyer">
      </div>
    </form>
  </body>
</html>
```

Pour l'analyser sans trop aller en détails, on voit qu'il est constitué de 3 parties.

Il y a la "form", qui est le plan de la requête. Ici on lui dit que c'est un POST et qu'il doit tout envoyer a form.php.

Ensuite, il y a les “input”. Ce sont les champs de données que l'utilisateur doit remplir.

Enfin, il y a le dernier “input”, qui est différent car il s'agit d'un bouton pour envoyer.

6. Structure d'une réponse (status code, headers, ...)

```
HTTP/1.1 200 OK
Date: Tue, 04 Nov 2025 15:10:45 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 1270
Connection: keep-alive
Server: nginx/1.18.0
Cache-Control: max-age=3600

<!DOCTYPE html>
<html>
<head>
  <title>Ma Page Web</title>
</head>
<body>
  <h1>Bonjour !</h1>
  <p>Ceci est le contenu de la page.</p>
</body>
</html>
```

Nous avons ici un exemple de réponse. Il y a également 3 partie.

Status-line.

C'est juste la première ligne. Elle indique la version http, le code de status (voir plus tard dans doc) et un message textuel pour l'utilisateur. Ici, 200 signifie OK.

Headers.

Déjà expliqués je reviendrai dessus un peu plus tard mais c'est comme une requête.

Body.

C'est basiquement ce que l'utilisateur a demandé. Ici, un html.

7. Définition des headers les plus importants :

Nous avons vu qu'il y avait différents types de headers utilisés tant dans les requêtes que dans les réponses. Pour les connaître, il suffit de visiter ce site qui les répertorie et les explique :

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers>

Je ne vais pas tous les décrire, il y en a trop. Mais pour en définir 2,3 très populaire/importants en sécurité, il y a:

X-Content-Type-Options : Il permet de se protéger de snifs MIME et force le navigateur à utiliser le content-type demandé.

Strict-Transport-Security : force l'utilisation de https, évidemment plus sûre.

X-Frame-Options : Bloque les Iframe, et protège donc des clicksacking.

Content-Security-Policy : Restreins les sources d'images, de scripts à travers des whitelist pour éviter les scripts malveillants.

Il y en a plein d'autres importants et fréquemment utilisés, il suffit de lire le site.

8. Définition des status codes les plus importants :

Comme vu précédemment les réponses comportent des codes de status. Encore une fois il y en a pleins. On peut tous les voir sur ce site :

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

On y voit qu'elles sont divisés en 5 catégories :

1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)

Les plus populaires étants :

200 OK Le succès classique : la requête a réussi.
201 Created Utilisé après un POST quand une nouvelle ressource est créée.
204 No Content La requête a réussi mais la réponse n'a pas de corps.
301 Moved Permanently Redirection permanente vers une nouvelle URL.
302 Found Redirection temporaire.
400 Bad Request Erreur côté client, syntaxe ou requête incorrecte.
401 Unauthorized Le client doit s'authentifier.
403 Forbidden Le client est identifié mais n'a pas les droits d'accès.
404 Not Found Ressource non trouvée.
500 Internal Server Error Erreur côté serveur.
503 Service Unavailable Le serveur est indisponible.

9. Définition des verbes les plus importants :

Il y a 9 méthodes de requêtes officielles définies en http/1.1 :

GET : Récupère une ressource.
HEAD : Identique à GET, mais sans le corps de la réponse.
POST : Envoie des données au serveur.
PUT : Remplace entièrement une donnée existante.
DELETE : Supprime une donnée.
CONNECT : Établit un tunnel réseau vers un serveur.
OPTIONS : Indique les méthodes supportées par le serveur ou la ressource.
TRACE : Retourne la requête reçue pour débogage.
PATCH : Met à jour partiellement une ressource.